tree in figure C.9 `is 3` because the longest path is `1->2->3->0` (or equivalently `1->2->3->9`).

In the next section, we will focus on introducing binary search trees.

C.1.4 Starting drag operations

A *binary tree* is a tree where the number of children for any node is at most 2.

A binary search tree (*BST*) is a binary tree where each node has a key associated with it, and satisfies two conditions: if `key(x)` is the key associated with a node `x`, then

- `key(x) >= key(y)` for every node `y` in the left subtree of `x`.
- `key(x) <= key(z)` for every node `z` in the right subtree of `x`.

A node's key can also be its value, but in general *BST*'s nodes can store a key and a value independently.

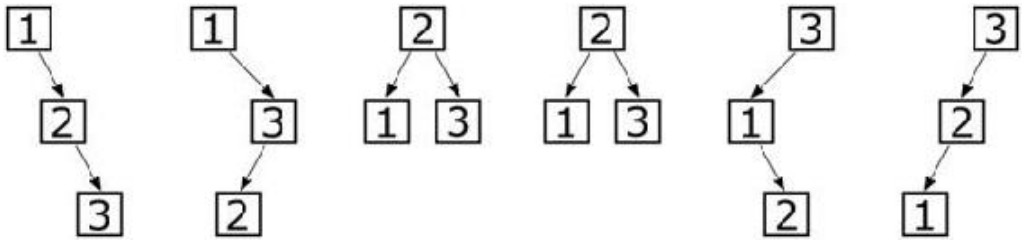


Figure C.10 All possible layouts for *BST*s of size 3. The layout depends on the order in which elements are inserted. Notice how 2 of the layouts are identical: for the insertion sequences `[2, 1, 3]` and `[2, 3, 1]`.

Let's continue with a few more definitions:

- A *balanced tree* is a tree where, for each node, the height of its left and right subtrees differs at most by 1, **and** both its left and right subtrees are balanced.
- A *perfectly balanced tree* is a balanced tree in which, for each internal node, the difference in the number of nodes of the left and right subtrees is at most 1.